

Quantitative Portfolio Manager: Design Specification

1. Executive Summary & Design Philosophy

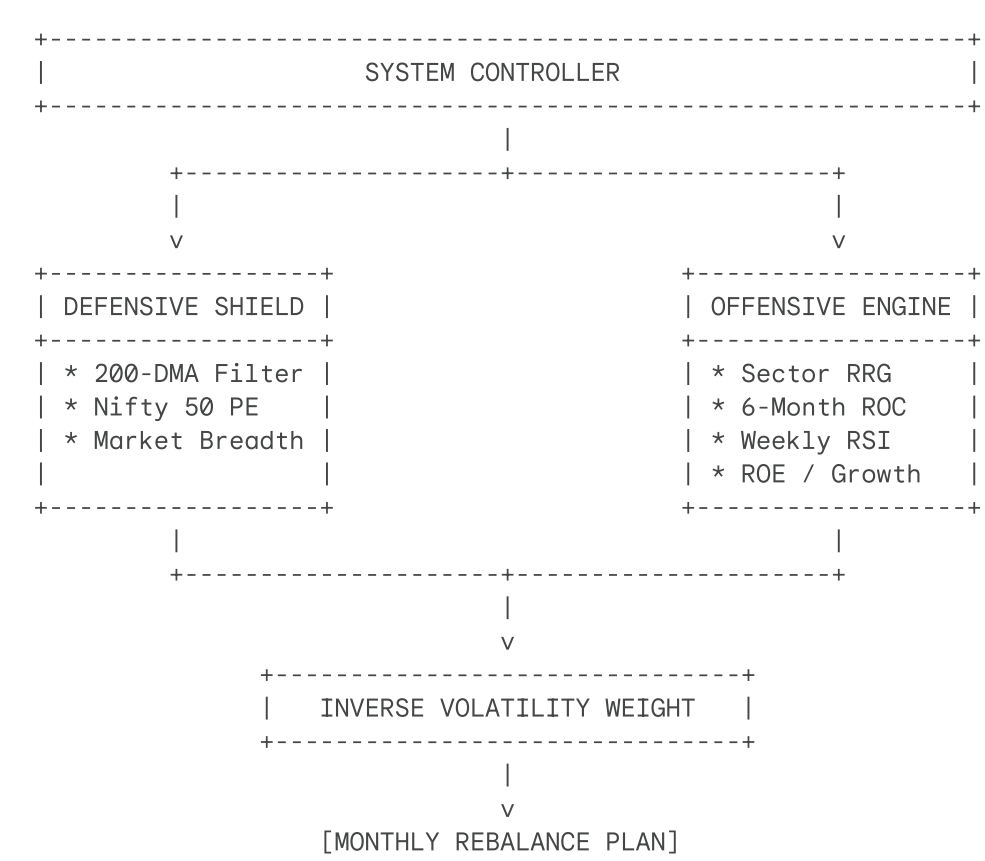
This document outlines the architecture for a non-trading, **Semi-Automatic Quantitative Portfolio Manager** optimized for the Indian financial market. Designed for an aggressive **5-to-10-year horizon**, the system manages a multi-asset portfolio comprising Indian Equities, Exchange-Traded Funds (ETFs), Mutual Funds (MFs), and Fixed Deposits (FDs).

Core Philosophy

- No Intra-day/Daily Trading:** Positions are held for mid-to-long-term horizons (3 to 12 months) to capture structural trends and business earnings acceleration.
- Objective Rule-Based Decisions:** Eradicates emotional biases by relying entirely on mathematically proven factors: Momentum, Quality, Value, and Regime Switching.
- Active Downside Protection:** Automatically transitions to safe assets (FDs/Liquid ETFs) when broad market trends fail, ensuring capital is preserved during severe drawdowns.
- Low Overhead Maintenance:** The investor interacts with the system exactly once a month for 20-30 minutes to review generated rebalance recommendations and execute them in one click.

2. Core Quantitative Strategies

The system runs on a dual-engine architecture: the **Defensive Shield** determines the maximum risk the portfolio is allowed to take, while the **Offensive Allocator** selects the best-performing assets to fill that risk bucket.



A. The Defensive Shield (Risk Management)

Before buying any stock, the system evaluates three macro health metrics to decide if the broader market is safe:

1. Market Regime (200-DMA):

- **Bullish:** Nifty 500 is above its 200-DMA. The system allocates 80%–100% of capital to equities/MFs.
- **Bearish:** Nifty 500 is below its 200-DMA. The system triggers defensive protocols, capping equity exposure at 50% and sweeping the rest to FDs or overnight mutual funds.

2. Market Valuation (Nifty PE Ratio):

- **Overvalued (PE > 24):** The script pauses new equity deployments and funnels SIPs directly into fixed-income instruments.
- **Fair Value (PE 18–22):** Normal operations continue.
- **Undervalued (PE < 16):** The script triggers "aggressive buybacks," drawing cash from FDs to deploy into highly ranked equities.

3. Internal Market Health (Market Breadth):

- Tracks the *Percentage of Nifty 500 Stocks trading above their 200-DMA*.
- If Nifty is hitting new highs but breadth is declining (divergence below 50%), the script issues a warning and restricts aggressive stock additions.

B. The Offensive Engine (Asset Selection)

When the Defensive Shield is clear, the Offensive Engine selects mid-term investments:

1. Sectoral Rotation via Relative Rotation Graphs (RRG):

- Computes the relative strength and momentum of sectoral indices against the Nifty 500.
- Directs capital *only* to stocks belonging to sectors currently in the **Leading** quadrant or fast ascending through the **Improving** quadrant.

2. Structural Momentum (6-Month ROC + Weekly RSI):

- Identifies stocks with strong upward trajectory.
- **Buy Rule:** 6-Month Rate of Change (ROC) > 20%, and Weekly RSI is between 60 and 75 (healthy trend acceleration, not yet overbought).
- **Sell Rule:** Individual stock drops below its personal 200-DMA or Weekly RSI falls below 40.

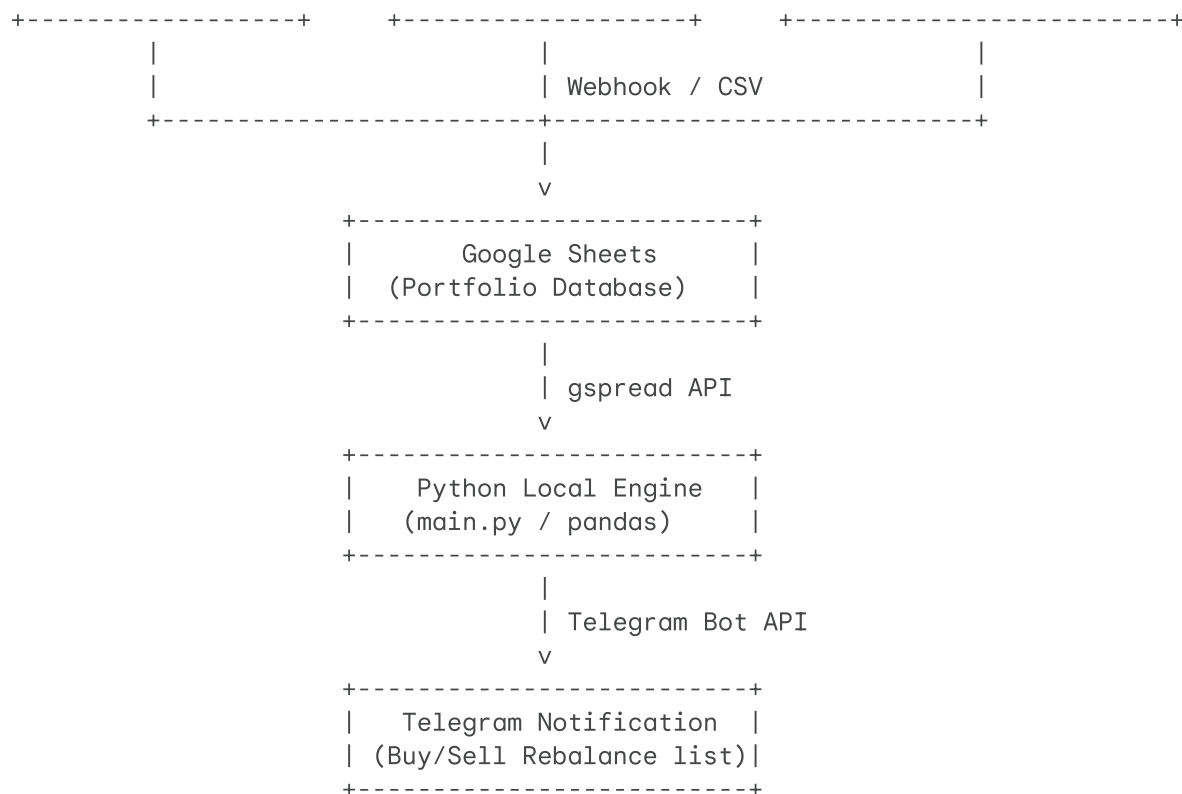
3. Quality & Earnings Acceleration Filter:

- Eliminates speculative companies.
- Stocks must have **Return on Equity (ROE) > 15%**, **Debt-to-Equity < 0.5**, and **Quarterly EPS growth showing acceleration** over the past two consecutive quarters.

3. Data & Technology Stack (The Hybrid Model)

To avoid the high costs of institutional APIs or the fragility of writing custom web scrapers, the system leverages a hybrid approach: external financial platforms clean the raw financial data, and Python serves as the decision engine.

+-----+	+-----+	+-----+
Screener.in	Chartink.com	Value Research Online
(Quarterly EPS/	(Momentum/ETFs/	(Mutual Fund Rolling
ROE/Quality)	Market Breadth)	Returns & Alpha)



- **Data Aggregation:**
 - **Screener.in:** Processes fundamental screening and quarterly balance sheets.
 - **Chartink.com:** Computes weekly RSI, volume breakouts, and 125-day (6-month) ROC.
 - **Value Research / Morningstar:** Monitors Mutual Fund performance.
- **Storage Engine:** Google Sheets API. Acts as an accessible, easily editable visual database for current holdings and transaction logs.
- **Decision Engine:** Python 3.x using `pandas` , `pandas-ta` , `gsread` , and `requests` libraries.
- **Communication:** Telegram Bot API for pushing rebalance plans directly to your mobile phone.

4. Database Architecture (Google Sheets Schema)

The Google Sheet acts as the central state tracker of the portfolio. The Python engine reads from and writes to this sheet.

Tab 1: Holdings

Tracks the assets you currently hold. This is the visual interface of your portfolio.

Ticker	Company Name	Asset Class	Quantity	Avg Buy Price	Current Price
TCS	Tata Consultancy	Stock	50	3800.00	=GOOGLEFINANCE ("NSE : TCS")
JUNIORBEES	Nippon India Nifty Jr	ETF	120	510.00	=GOOGLEFINANCE ("NSE : JUNIORBEES")

PARAGPARIKH	PPFAS Flexi Cap Fund	Mutual Fund	1500	72.50	<i>Updated via script</i>	
HDFCFD_01	HDFC 1-Year FD	FD	1	150000.00	150000.00	

Tab 2: Ledger

A permanent log of every historical buy/sell transaction. This allows Python to track portfolio changes over time.

Date	Ticker	Asset Class	Action	Quantity	Execution Price	Total Value	Charges & Taxes
2026-06-05	TCS	Stock	BUY	50	3800.00	190000.00	250.00
2026-07-12	INFOSYS	Stock	SELL	100	1550.00	155000.00	180.00

Tab 3: Signals

The raw scratchpad where your external screening webhooks or CSV imports place candidate assets for the month.

Date	Ticker	Strategy	ROC_6M	RSI_Weekly	ROE	Sector
2026-07-12	TRENT	Momentum+Quality	45.2%	68.4	22.3%	Retail
2026-07-12	TATAMOTORS	Momentum+Quality	28.1%	61.2	18.5%	Auto

5. Python Software Architecture

The Python program is completely modularized. Each major task is separated into its own class so that you can add or change features (like connecting a broker API later) without breaking the rest of the application.

```
# =====
# MASTER ENGINE BLUEPRINT: main.py
# =====

import gspread
import pandas as pd
import pandas_ta as ta
import yfinance as yf
import requests

class PortfolioTracker:
    """Manages reading and writing to the Google Sheets visual database."""
    def __init__(self, sheet_id):
        self.sheet_id = sheet_id
        # Initialize Google Sheets connection (gspread)
```

```

def get_current_holdings(self):
    """Returns a Pandas DataFrame of the 'Holdings' sheet."""
    pass

def add_to_ledger(self, date, ticker, action, qty, price):
    """Logs an executed trade to the 'Ledger' sheet."""
    pass

class MarketRegime:
    """Analyzes overall market health, trend, and valuations."""
    def __init__(self):
        self.nifty_ticker = "^CNX500" # Nifty 500 Index

    def get_market_trend(self):
        """
        Downloads Nifty 500 historical data.
        Returns: 'BULLISH' if Close > 200-DMA, otherwise 'BEARISH'
        """
        data = yf.download(self.nifty_ticker, period="1y")
        close = data['Close'].iloc[-1]
        dma_200 = data['Close'].rolling(window=200).mean().iloc[-1]
        return "BULLISH" if close > dma_200 else "BEARISH"

    def get_nifty_pe_ratio(self):
        """
        Fetches current Nifty PE Ratio.
        If PE > 24: Returns 'OVERVALUED'
        If PE < 16: Returns 'UNDERVALUED'
        Else: Returns 'FAIR'
        """
        pass

class SignalProcessor:
    """Matches candidate stocks from external screeners with internal filters."
    def __init__(self, raw_signals_df):
        self.signals = raw_signals_df

    def filter_candidates(self):
        """
        Applies mathematical filters:
        - 6-Month ROC > 20%
        - Weekly RSI in [60, 75]
        - ROE > 15%
        Returns list of approved tickers.
        """
        pass

class AllocationEngine:
    """Main brain. Compares current holdings to target states and creates order
    def __init__(self, holdings, regime, approved_signals):
        self.holdings = holdings
        self.regime = regime
        self.approved_signals = approved_signals

    def generate_rebalance_orders(self):
        """
        1. If Regime is BEARISH: Recommend selling weak stocks to move to 50% F
        2. If Regime is BULLISH:
            - Identify current holdings with Momentum loss (Weekly RSI < 40) ->
            - Allocate released cash + new SIP capital to top Approved Signals.
            - Use Inverse Volatility to assign safe weights to each asset.
        """

```

```

        orders = []
        # Calculate volatility-adjusted sizing
        return orders

class TelegramNotifier:
    """Sends clean, readable transaction instructions straight to your mobile."""
    def __init__(self, bot_token, chat_id):
        self.token = bot_token
        self.chat_id = chat_id

    def send_notification(self, message_text):
        """Sends HTTP POST to Telegram Bot API."""
        url = f"https://api.telegram.org/bot{self.token}/sendMessage"
        payload = {"chat_id": self.chat_id, "text": message_text, "parse_mode":
requests.post(url, json=payload)

# =====
# SYSTEM COORDINATION RUNNER
# =====
def run_monthly_rebalance(sheet_id, telegram_token, telegram_chat_id):
    # 1. Initialize Tracker and fetch state
    tracker = PortfolioTracker(sheet_id)
    holdings = tracker.get_current_holdings()

    # 2. Analyze macro market regime
    regime_analyzer = MarketRegime()
    trend = regime_analyzer.get_market_trend()
    valuation = regime_analyzer.get_nifty_pe_ratio()

    # 3. Read raw monthly inputs from Chartink & Screener.in
    raw_signals = tracker.get_raw_signals()
    processor = SignalProcessor(raw_signals)
    approved_investments = processor.filter_candidates()

    # 4. Generate orders
    allocator = AllocationEngine(holdings, {"trend": trend, "valuation": valuat
rebalance_plan = allocator.generate_rebalance_orders()

    # 5. Push to user's mobile device
    messenger = TelegramNotifier(telegram_token, telegram_chat_id)
    messenger.send_notification(rebalance_plan)

if __name__ == "__main__":
    # Scheduled to run on the 1st Saturday of every month
    run_monthly_rebalance("your_google_sheet_id", "your_bot_token", "your_chat_

```

6. Monthly Operational Workflow

Running this system requires only four simple, non-technical steps on the first weekend of every month:

1. **Auto-Signals (10 Minutes):** Run your saved screeners on Screener.in and Chartink. Copy the output list of tickers and paste them directly into the `Signals` tab of your Google Sheet.
2. **Execute Python Decision Engine (2 Minutes):** Double-click the Python script (or let it run automatically on PythonAnywhere).
3. **Review Recommendation (3 Minutes):** Read the structured message that arrives on your Telegram app.

4. **Execute (10 Minutes):** Open your broker app (e.g., Zerodha, Groww). Execute the few buy/sell trades recommended, modify your SIP amounts if the defensive shield was triggered, and update your Google Sheet holdings to match the new allocations.

7. Future-Proofing Roadmap

The modular, object-oriented design makes it incredibly simple to scale this tool over time:

[Phase 1: Semi-Automatic] ---> [Phase 2: Live Tracking Dashboard] ---> [Phase 3
(Telegram & Manual Execution) (Flask/React visual analytics UI) (Dire

- **To add Automated Live Execution:** Create a new Python module called `BrokerAPI`. Pass the orders outputted from the `AllocationEngine` into a library like `KiteConnect` to execute the trades automatically without manual broker logging.
- **To add a Tax-Loss Harvesting Engine:** Modify the `PortfolioTracker` to fetch the buy-date from the Ledger sheet. Code a script that identifies short-term losses at year-end, sells them to book tax benefits, and instantly replaces them with a highly correlated ETF.
- **To add Real-time Analytics:** Construct a simple local Python webpage using the **Streamlit** library. Point it to read the Google Sheets file to display beautiful visual dashboards tracking your returns against the Nifty 50.